

5-stage Pipelined RISC-V CPU

Rohtak Patwardhan, all rights reserved

Summer 2026

Contents

1	Introduction	3
2	Architecture Overview	3
2.1	R-Type Format	3
2.2	I-Type Format	3
2.3	B-Type Format	4
2.4	S-Type Format	4
2.5	J-Type Format	4
3	Instruction Set	5
3.1	R-Type Instructions (7 instructions)	5
3.2	I-Type Instructions (4 instructions)	5
3.3	S-Type Instructions (1 instruction)	5
3.4	B-Type Instructions (2 instructions)	5
3.5	J-Type Instructions (2 instructions)	5
4	Register File	6
4.1	Read Access Interface	6
4.2	Write Access Interface	6
5	Memory Model	6
6	Design Rationale	6
7	Summary	6

1 Introduction

This custom processor uses a 16-instruction subset of the popular RISC-V 32-bit processor instruction set architecture. For more information on the RV32I, please refer to the official RISC-V documentation. The reason I undertook this project is to gain experience and provide an extension to RISC-V CPUs. Additionally, I wanted to immerse myself more into FPGA and VLSI while learning about power management integration, accelerated 5-stage cycles, and RV vector processing.

2 Architecture Overview

The ISA consists of 5 types of instructions, namely R-type, I-type, J-type, B-type, and S-type.

2.1 R-Type Format

R-type instructions are register-to-register, meaning that the source operands and destination operand are registers. The operations work on the data contents of the registers themselves, placing the result into a final register. These use two source operands (rs1, rs2) and one final destination register (rd).

Field	Width (bits)
funct7	7
rs2	5
rs1	5
funct3	3
rd	5
opcode	7

2.2 I-Type Format

This format is used for operations that operate on immediate formats, or sign-extended constant values. It uses a source and destination register, as well as a 12-bit immediate, sign-extended, operand representing a constant value that can range from -2^{11} to $2^{11} - 1$.

Field	Width (bits)
imm[11:0]	12
rs1	5
funct3	3
rd	5
opcode	7

2.3 B-Type Format

Branch type instructions, which allow for stronger control flow and conditional logic. These instructions compare two registers and use a 12-bit value to identify a relative jump address. The 12-bit immediate represents a signed byte offset from the current program counter, or PC.

Field	Width (bits)
imm[12]	1
imm[10:5]	6
rs2	5
rs1	5
funct3	3
imm[4:1]	4
imm[11]	1
opcode	7

2.4 S-Type Format

Store instructions, allowing for values to be stored into registers or memory locations given by a unique address.

Field	Width (bits)
imm[11:5]	7
rs2	5
rs1	5
funct3	3
imm[4:0]	5
opcode	7

2.5 J-Type Format

Jump instructions. Unconditional jumps. JAL, jump and link.

Field	Width (bits)
imm[20]	1
imm[19:12]	8
imm[11]	1
imm[10:1]	10
rd	5
opcode	7

3 Instruction Set

3.1 R-Type Instructions (7 instructions)

Instruction	funct3	funct7	Type	Operation	Example
ADD	0x0	0x00	Arithmetic	$rd = rs1 + rs2$	add x1, x2, x3
SUB	0x0	0x20	Arithmetic	$rd = rs1 - rs2$	sub x1, x2, x3
SLL	0x1	0x00	Shift	$rd = rs1 \ll rs2$	sll x1, x2, x3
SRL	0x5	0x00	Shift	$rd = rs1 \gg rs2$	srl x1, x2, x3
OR	0x6	0x00	Logic	$rd = rs1 \mid rs2$	or x1, x2, x3
AND	0x7	0x00	Logic	$rd = rs1 \& rs2$	and x1, x2, x3
XOR	0x4	0x00	Logic	$rd = rs1 \hat{\ } rs2$	xor x1, x2, x3

3.2 I-Type Instructions (4 instructions)

Instruction	Opcode	funct3	Operation	Example
ADDI	0010011	0x0	$rd = rs1 + imm$	addi x1, x2, 100
ORI	0010011	0x6	$rd = rs1 \mid imm$	ori x1, x2, 0x10
ANDI	0010011	0x7	$rd = rs1 \& imm$	andi x1, x2, 0xFF
LW	0000011	0x2	$rd = MEM[rs1 + imm]$	lw x1, 0(x2)

3.3 S-Type Instructions (1 instruction)

Instruction	funct3	Operation	Example
SW	0x2	$MEM[rs1 + imm] = rs2$	sw x1, 0(x2)

3.4 B-Type Instructions (2 instructions)

Instruction	funct3	Condition	Operation	Example
BEQ	0x0	Equal	if ($rs1 == rs2$) $PC += imm$	beq x1, x2, label
BLT	0x4	Less Than	if ($rs1 < rs2$) $PC += imm$	blt x1, x2, label

3.5 J-Type Instructions (2 instructions)

Instruction	Opcode	Operation	Example
JAL	1101111	$rd = PC + 4; PC += imm$	jal x1, label
JALR	1100111	$rd = PC + 4; PC = rs1 + imm$	jalr x1, x2, 0

4 Register File

The processor has 32 general-purpose registers. x0 is hardwired to zero. x1-x31 are used to hold intermediate data, addresses, and results of operations. There are also 32 optional floating-point registers ranging from f0-f31.

The register file consists of two interfaces:

4.1 Read Access Interface

Data is read from the decode unit and sent to the execution unit. This interface is used to read the register at the address sent by the decode unit. If the operation requires two source registers, then two read ports are required at the read access interface to read two registers at the same time. Single-cycle access is provided to allow the read data to be sent to the execution unit within the same clock cycle.

4.2 Write Access Interface

The write access interface is used for writeback from the writeback unit. One write port is always sufficient because there is only one destination register (rd). Single-cycle access is provided, allowing the writeback unit to update the register at the address sent within a single clock cycle.

5 Memory Model

Memory consists of 32-bit addressable locations, byte-addressed, with little-endianness. Little-endianness means that the least significant byte (LSB) is at the lower end of the address.

6 Design Rationale

I chose this specific subset of the ISA because I wanted to achieve a universal implementation, which was possible using the AND, OR, and XOR operations. Additionally, I wanted to be able to do general arithmetic, and being able to implement NOT using XOR allowed me to be more concise with my jump instructions, leading me to choose two that would cover the most ground. Finally, I chose one load and store instruction for the capacity to use the instructions, and I chose immediate instructions for more extended functionality.

7 Summary

This 16-instruction RISC-V processor is a complete, Turing-complete implementation capable of executing any computable algorithm. The instruction set consists of 7 R-Type instructions for register-based computation, 4 I-Type instructions for immediate operations and memory loads, 1 S-Type instruction for stores, 2 B-Type instructions for conditional

branching, and 2 J-Type instructions for jumps and function calls. The combination of arithmetic operations, logical operations, memory access, and control flow makes this ISA universal.